Checklist de déploiement kiosk

Ce que le relecteur d'admission cert doit s'attendre à trouver sur la machine de test, de la clé USB au premier démarrage. Reflète l'image kiosque mkosi (`deploy/image/`) et le flux de provisioning en trois phases (`deploy/provision/provision.sh`), pas l'ancien installeur monolithique.

Les cases à cocher sont un **modèle** : elles se remplissent à l'installation de chaque terminal (et lors de la visite d'admission pour la machine de test), pas dans ce document.

Couche matériel

- ☐ Penetek A3216 M82 ou Advantech USC-360J (ou terminal x86_64 fanless équivalent avec écran tactile PCAP 15.6" 1920×1080).
- ☐ **TPM 2.0 activé en BIOS** (fTPM/Intel PTT ou puce discrète) - requis pour le chiffrement LUKS2 de `/var` au premier démarrage (`SECURITE_BD_FR.md` §3.6). Sans cela, `pili-provision.service` échoue et le terminal ne dépasse jamais l'écran de mots de passe.
- ☐ Secure Boot **désactivé** (aucune infrastructure de signature de l'UKI n'existe encore - l'activer empêche le boot).
- ☐ Imprimante thermique Epson TM-m30III raccordée au LAN (TCP 9100 ; l'USB n'est pas pris en charge).
- ☐ Tiroir-caisse RJ11 raccordé à l'imprimante.
- ☐ URL du FDM module2 (boîtier sur le LAN, ou simulateur en ligne `https://fdm.module2.be/graphql/` pour la machine de test) configurée dans `/etc/pili-pos/pos.json::fdm_url` (en `https://`), son jeton Bearer dans le fichier désigné par `fdm_token_file`, et les certificats module2 (`ca.crt`, `client.crt`, `client.key` en droits 600) dans `/etc/pili-pos/fdm/`, désignés par `fdm_ca_file`, `fdm_client_cert_file`, `fdm_client_key_file`.

Couche OS (image mkosi, `deploy/image/`)

- ☐ Debian 13 "trixie" minimal, construit par mkosi (`deploy/image/mkosi.conf`, profils `production/admission` sous `mkosi.profiles/`) - sans environnement de bureau.
- ☐ Disque GPT : ESP (512 M) + racine ext4 **lecture seule** (4 G, systemd-boot + UKI) + partition `pili-data` (tout l'espace restant, créée par `systemd-repart` au premier démarrage).
- ☐ `pili-data` chiffrée **LUKS2**, déverrouillage automatique via **TPM2** (PCR 7) + clé de récupération de secours, mise en place par `pili-provision.service` au tout premier démarrage uniquement (`SECURITE_BD_FR.md` §3.6).
- ☐ Mot de passe `root` aléatoire (et `spf` sur l'image d'admission) généré et affiché **une seule fois** à la fin du premier démarrage, avec la clé de récupération LUKS - à vérifier que le relecteur/distributeur les a bien notés (`MANUEL_INSTALLATION_FR.md` §3.4).
- ☐ Compositeur Cage Wayland - lance automatiquement `pili-pos` en plein écran au boot, sur `tty1` exclusivement (`Conflicts=getty@tty1.service`).
- ☐ Service systemd `pili-pos.service` activé, tourne sous l'utilisateur non-root `pos` (`SECURITE_BD_FR.md` §3.2).
- ☐ Heure système synchronisée via chrony (les horodatages fiscaux corrects sont une exigence cert).
- ☐ RustDesk pinné (SHA-256 vérifié), extrait avec `--appimage-extract` dans `/opt/pili-pos/rustdesk/` (`AppRun`) - aucun service, jamais lancé au boot (`deploy/REMOTE-OPS.md`).
- ☐ Pas de serveur SSH (appliance scellée), Bluetooth/CUPS/Avahi/ ModemManager masqués. Accès console maintenance uniquement sur `tty2` (`Ctrl+Alt+F2`), terminal physique, login `root`.
- ☐ USB installer/updater : le même artefact `.img.xz` propose au boot les entrées systemd-boot **"Install"** et **"Update"** (`pili.installer=install|update`) - "Install" réplique la table de partitions (`sgdisk --replicate`) et écrase le disque interne ; "Update" ne recopie que ESP+racine (dd partition à partition) et **préserve** `pili-data` (donc `pos.db`).

Couche application

- ☐ Build cert de `pili-pos` installée dans `/opt/pili-pos/`. Produite par `deploy/build-release.sh --docker` (build Linux x86_64, `--no-default-features --features cert`, UI embarquée) - le SHA-

1 déclaré est vérifié par `.github/workflows/image.yml` (job `repro`, build reproductible x2, puis job `image`, hash de `/opt/pili-pos/pili-pos` **à l'intérieur** de l'image finie, partition racine montée en lecture seule).

- ☐ Binaire `pos-fdm-mock` dans `/opt/pili-pos/bin/pos-fdm-mock` (utilisé pendant la démo d'admission si le FDM virtuel est injoignable) - image d'admission uniquement.
- ☐ Base de données de seed dans `/var/lib/pili-pos/pos.db`, initialisée à la construction de l'image (dans `/usr/share/pili-pos/factory/` puis copiée sur `/var` déchiffré au premier démarrage) avec `src-tauri/seed_spf_test.sql` (image d'admission) : - Lignes de produits prédéfinies du manuel §III.N (Dry Martini, Burger of the Chef, …) avec leurs productId et codes TVA, requises pour rejouer les UC tels que définis. - Le personnel du jeu de test (John, Elisa, Sammy, avec leurs employeeId) de `docs/20260821_UC_Reports_1.3/UC_TestSetup_20240806.md` (le dossier du Basic Set 1.8 ne contient plus ce fichier), plus un compte par rôle. - `pos.json` à l'identité du jeu de test (The FPS Finances Experience, `CPOS0031234567`, `POS-1-BAR`, 25 tables). - L'image de production (restaurant) utilise à la place le catalogue propre à l'établissement ; le binaire `pili-pos` est identique dans les deux images.
- ☐ `pos.json::device_id` renseigné automatiquement par `pili-provision` (adresse MAC du terminal, pas `CHANGE_ME_MAC`) - voir `MANUEL_INSTALLATION_FR.md` §4.2.
- ☐ IP LAN de l'imprimante configurée dans `pos.json::printer_address` (et, si le bar ou la cuisine a sa propre imprimante, `pos.json::kitchen_printer_address` : voyant "Cuisine" vert).
- ☐ Sur l'image d'admission : compte `spf` (groupe `pos`, `tty2`, mot de passe affiché une seule fois au premier démarrage) peut exécuter `pos-db-ro` pour une lecture seule de `pos.db` sans jamais avoir besoin d'un accès écriture (`FEUILLE_ACQUISITION_DONNEES_FR.md` §2.2).

Documentation dans la clé USB du dossier

Les documents ci-dessous sont ceux du dossier de soumission (`MANIFEST.txt`) ; chacun est fourni en PDF.

- ☐ Réponses au manuel de certification : `REPONSES_MANUEL_CERT_FR`, `PROFIL_ENTREPRISE_FR`, `LISTE_DISTRIBUTEURS_FR`, `DECLARATION_FONCTIONS_INTERDITES_FR`, `LISTE_FONCTIONNALITES_FR`
- ☐ Manuels : `MANUEL_UTILISATEUR_FR`, `MANUEL_INSTALLATION_FR`, `MANUEL_PROGRAMMATION_FR`, `PROFILS_UTILISATEURS_FR`, `CAPTURES_ECRAN_FR`
- ☐ Architecture et données : `DOCUMENT_PROCESSUS_TECHNIQUE_FR`, `FLUX_DONNEES_FR`, `SCHEMA_BD_FR`, `SCHEMA_ER_FR`, `SECURITE_BD_FR`, `FEUILLE_ACQUISITION_DONNEES_FR`, `SCHEMA_ASSEMBLAGE_FR`, `CHECKLIST_DEPLOIEMENT_KIOSK_FR`, `GARANTIE_MAJ_LANGUES_FR`
- ☐ Fiscalité et conformité : `MAPPING_CONFORMITE_FR`, `STRATEGIE_COMPTEURS_FR`, `FLUX_REMBOURSEMENT_FR`, `LIGNES_PRODUITS_FR`, `TICKETS_EXEMPLES_FR`, `DECISION_PEPPOL_AP_FR`, `TICKET_NUMERIQUE_FR`, `COMMANDES_EN_LIGNE_FR`
- ☐ Tests : `RAPPORT_TESTS_BASELINE_FR`, `COUVERTURE_FIXTURES_FR`, `COUVERTURE_RAPPORTS_FR`, `COUVERTURE_ADVANCED_SET_FR`, `RESULTATS_TESTS_UC_FR`, `JOURNAL_TESTS_PRE_SOUMISSION_FR` (rempli après la répétition de Phase 11)
- ☐ Preuves : dossier `Report_Testing` (requêtes et réponses JSON, photos ou PDF des tickets) et captures d'écran (`screenshots`)
- ☐ Formulaire de demande de certification signé (PDF + copie scannée signée).

Commandes de vérification (sur la machine de test)

L'image kiosk de production embarque **sans code source ni toolchain Rust** - uniquement le binaire signé, la base SQLite et le service systemd. Les commandes de vérification ci-dessous s'exécutent sur la **machine de build du développeur** (un poste de travail séparé), contre l'arbre source qui a produit le binaire, pas sur le kiosk.

```
# Sur la MACHINE DE BUILD (arbre source présent), avant la production
# de l'image kiosk :
cd ~/pili-pos                          # le dépôt source

# 1) Build cert reproductible (linux/amd64, Docker) + rapport SHA-1.
./deploy/build-release.sh --docker
```

```
# → deploy/dist/pili-pos, deploy/dist/build_info_*.txt

# 2) Confirmer le périmètre du binaire cert : commandes en ligne
#    présentes (activées par installation), facturation B2B absente.
strings target/linux-amd64/release/pili-pos | grep -c integration_switch_log   # → > 0
nm target/linux-amd64/release/pili-pos | grep -c pos_peppol                    # → 0

# 3) Confirmer que fiscal_events est append-only (pas de DELETE/UPDATE
#    dans le code compilé dans le binaire ; scripts/build_and_hash.sh,
#    étape 3/5, fait la même vérification). Les dossiers tests/ des
#    crates (tests d'intégration, hors binaire) sont exclus.
grep -rn --exclude-dir=tests "DELETE FROM fiscal_events\|UPDATE fiscal_events" \
  src-tauri/src/ crates/          # → uniquement des lignes de blocs #[cfg(test)]
                                  #   (crates/pos-fiscal/src/archive.rs,
                                  #   crates/pos-db/src/migrate.rs)

# 4) Lancer la suite de tests hors-ligne.
cargo test --workspace --no-fail-fast                          # → tout vert

# 5) Journée d'exploitation contre le simulateur module2 (URL et jeton
#    dans .env.live, cf. docs/internal/LIVE_FDM_MODULE2.md).
scripts/live_fdm.sh
# → tous les UC passent ; chaque réponse passe le contrôle de cohérence

# 6) Construire l'image kiosque (les deux profils) - root Linux x86_64
#    réel, ou conteneur linux/amd64 privilégié depuis macOS.
sudo deploy/image/build.sh production admission
# → deploy/image/mkosi.output/<profil>/pili-pos-kiosk-<profil>.img.xz
```

Sur le **kiosk lui-même**, une fois démarré, le relecteur peut vérifier que la version en cours d'exécution correspond au hash du dossier :

```
sha1sum /opt/pili-pos/pili-pos
# → doit correspondre à la valeur déclarée dans le formulaire cert
#   et dans deploy/dist/build_info_*.txt
```

`.github/workflows/image.yml` automatise les étapes 1 et 6 en CI : job `repro` (build #1/#2, compare les SHA-1), job `image` (build les deux profils, vérifie sur la partition racine montée en lecture seule que le binaire embarqué dans chaque `.img.xz` a bien le SHA-1 certifié), job `boot-smoke` (premier démarrage complet de l'image d'admission sous QEMU+KVM+swtpm : provisionnement LUKS2 + TPM2, écran des mots de passe validé par Entrée, puis kiosque ; échoue si un service a échoué, si un cycle d'ordonnancement a été rompu ou si `pili-pos.service` ne reste pas démarré, d'après le journal du système invité copié sur son port série ; capture d'écran finale et journal publiés en artefact CI `boot-smoke-screenshot`).

Script de démonstration pour le rendez-vous d'admission

0. **Avant le rendez-vous** : vérifier que le premier démarrage a déjà été fait sur CE terminal (LUKS2/TPM2 provisionné, mots de passe notés - §3.4 du manuel d'installation) ; le rendez-vous d'admission ne doit pas être le moment où l'assistant découvre l'écran de mots de passe à usage unique.
1. Boot du kiosk ; montrer Cage qui lance pili-pos en plein écran.
2. Login (tuile + PIN) ; montrer l'événement fiscal **S** work-in archivé.
3. Ouvrir Table 4 ; ajouter trois articles ; "Envoyer en cuisine" - événement **P** et bon de commande imprimé (sans prix).
4. Payer la table en deux parts égales ("Partager en" 2) : une part en espèces, l'autre par carte - **un seul** événement **N** (un ticket, deux paiements). Si l'installation a un terminal de paiement relié (`payment_terminal`), payer la part carte sur ce terminal (fenêtre "Paiement sur le terminal", marque de la carte imprimée sur le ticket) ; sans terminal relié, la carte se saisit à la main.

5. Rapports, onglet “Tickets” : “Copie” du ticket de la vente - événement **C** avec `fdmRef`.
6. Rapports, onglet “Journée” : “Rapport X” - événement **R**.
7. “Clôturer la journée (Z)”, puis “Confirmer la clôture ?” - événements **R**+**R**.
8. Écran de connexion : tuile, “Fin de service”, PIN - pointage sortie, événement **S**.
9. Inspecter `fiscal_events` en lecture seule (`pos-db-ro` sur le compte `spf`, ou `sqlite3 -readonly` sur la console `root`) - compteurs strictement monotones par label ; chaque ligne a un `fdm_response_json` non-null.
10. Démontrer le blocage en cas de FDM déconnecté en tuant le mock FDM en plein milieu d’un flux.